

Necessarily Rank-Maximal Matching

RASC based elicitation

ANALYSIS and SIMULATION

Madhav P Nair - 112301020
Devapriya Pradeep - 112301006

May 10, 2026

Matching under preferences



Preference table

- $\text{agents}(N) = \text{individuals}$.
- $\text{objects}(O) = \text{candies}$.

Agents	Preferences			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

Match them

Arbitrary Matchings

Agents	Preferences			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

Agents	Preferences			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

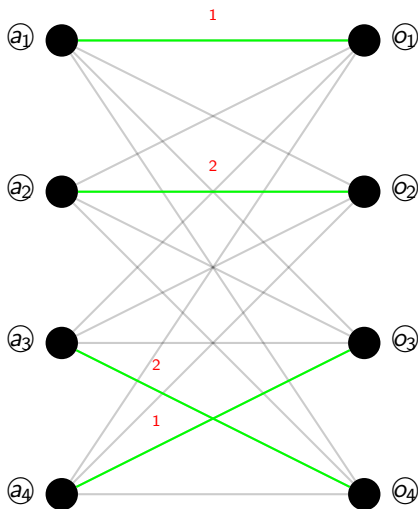
Which of these two matchings is better?

Goal : **Optimal Resource Allocation**

Rank-Maximal Matching

- An optimality criterion in resource allocation
- The maximum people get their first choice (rank), subject to that maximum people getting their second choice (rank), and so on!
- This n-tuple $(x_1, x_2, \dots, x_n)$ called **Signature of the Matching** is maximized in RMM
 - here x_i is the number of people getting i^{th} choice

Rank Maximal Matching



Signature = (2,2,0,0)

Necessarily Rank-Maximal matching

Motivation

Consider an instance where both N and O is very large, say $|N| = |O| > 100$. It is impractical to ask the full preferences of every agent.

Agents	Preferences			
a_1	o_1	o_2	$\dots$	$\dots$
a_2	o_1	o_2	$\dots$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
a_n	o_3	o_4	$\dots$	$\dots$

Goal

Compute a rank-maximal matching from **partial preference lists** of the agents that remains a rank-maximal matching under **all possible completions** of the partial preference lists.

Online Elicitation of NRM

Agents	Preferences		
a_1			
a_2			
a_3			

Agents	Preferences		
a_1	o_1		
a_2	o_1		
a_3	o_2		

Agents	Preferences		
a_1	o_1	o_3	
a_2	o_1	o_2	
a_3	o_2	o_3	

- Start with an empty preference list and query agents
- Agents' preferences are elicited only when needed in an online manner.
- This online variant is called Online NRM Elicitation
- Different query protocols queries differently

Upper and Lower bounds for the best algorithms for NRM Elicitation

	Next-Best		Set-Compare		Hybrid	
	LB	UB	LB	UB	LB	UB
NRM	$\frac{3}{2}$	$\frac{3}{2}$	$\frac{3}{2}$	$O(n)$	$\frac{3}{2}$	6

Table: Comparison table with LB and UB under each query model

Upper Bound for Set Compare Model (Peters, 2022)

Find an online algorithm with a **constant or sublinear competitive ratio** for eliciting a *necessarily rank-maximal (NRM) matching* under the *Set-Compare query model*.

Rank-Aware Set Compare Model (RASC)

Rank Aware Set Compare Model

The model asks the agent to reveal not just the most preferred object in the set but also the rank of the selected object:

$$Q(a_i, O') = (r, o), \quad O' \subseteq O$$

$$o = \arg \max_{o' \in O'} \text{rank}(a_i, o), \quad k = \text{rank}(a_i, o)$$

with respect to $\succ_i$, the complete preference list of agent a_i .

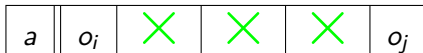
Our Algorithm

- Maintain agent-specific query subsets. **Delete inactive objects** from them in each iteration
- Each query to an agent retrieves both her next best available choice and its rank
- Store the (object, rank) pair in an **in-memory dictionary** if rank $>$ current iteration number
- Let the agent retire till this stored preference is used - **SAVE QUERIES**
- Produce an NRMM in reduced number of queries

Key Observations

No more futile queries to get inactive posts

- Maintaining agent-specific query subsets of only available objects by deleting her revealed posts and inactive posts
- If an agent returns a higher rank object in an iteration, the objects in between the last revealed rank and this higher rank are already inactive for this agent

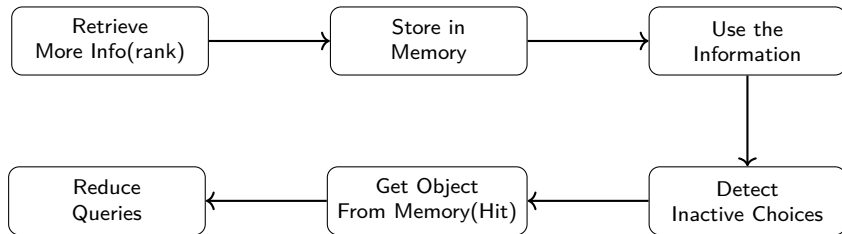


Dictionary

$\{a:(o_j, 5)\}$

- Let a retire till $j + 1^{th}$ iteration

Optimization



- We quantify the performance of the online algorithm by calculating the **competitive ratio**

$$\alpha = \frac{ALG}{OPT} = \frac{\text{No.of queries asked by the algorithm}}{\text{No.of queries asked by the optimal algorithm}}$$

Approach : Agent-specific

- Consider any agent a and compare the query cost of ALG and OPT
- Impose a lower bound constraint on OPT w.r.t ALG
- Prove by contradiction that OPT can be failed (gives non - NRM) if the constraint is violated

Analysis

We used the following lemmas to analyze our algorithm:

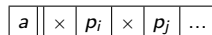
Critical Rank Lemma

Let G be a bipartite graph, with $(a, p) \notin E(G)$, where both a and p are even. Let $H = G \cup \{(a, p)\}$. Then, for every $1 \leq i < c$, the edge (a, p) belongs to **every** rank-maximal matching of H_i . Critical rank is defined as the first iteration where either a or p turns odd or unreachable.

Swapping two posts

If we swap positions of two posts p_i and p_j , both belonging to $rev(a)$, where $rank(a, p_i) < rank(a, p_j)$ then after the swap a will be matched to p_j in **every** RMM.

- p_j is always even before or at $rank(a, p_i)$
- critical rank of $(a, p_j) \geq rank(a, p_i) + 1$



Preference Manipulation by Swapping

- If OPT matches a to a revealed(OPT) object, we swap any two unrevealed(ALG) objects

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\textcircled{o_3}$	$\times$	$\textcircled{o_5}$	$\textcircled{o_6}$	$\dots$	o_k

swap

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\textcircled{o_5}$	$\times$	$\textcircled{o_3}$	$\textcircled{o_6}$	$\dots$	o_k

- If a is matched to o_3 (unrevealed by OPT), we swap o_3 and o_5

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\textcircled{o_3}$	$\times$	$\textcircled{o_5}$	$\textcircled{o_6}$	$\dots$	o_k

swap

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\textcircled{o_5}$	$\times$	$\textcircled{o_3}$	$\textcircled{o_6}$	$\dots$	o_k

Preference Manipulation by Swapping

- If a is matched to o_5 (unrevealed), then we swap o_5 and o_6

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\odot_3$	$\times$	$\odot_5$	$\odot_6$	$\dots$	o_k

swap

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\odot_3$	$\times$	$\odot_6$	$\odot_5$	$\dots$	o_k

- If a is matched to o_6 (unrevealed), we swap o_5 and o_3

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\odot_3$	$\times$	$\odot_5$	$\odot_6$	$\dots$	o_k

swap

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$\dots$	$r = k$
a	o_1	$\times$	$\odot_5$	$\times$	$\odot_3$	$\odot_6$	$\dots$	o_k

- The promoted object is matched with a in every RMM
- OPT fails to give an NRM when $OPT < ALG - 2$

Constraint on optimal algorithm

Optimal algorithm must reveal at least $q_a - 2$ objects for an agent a that is matched to a revealed post, where q_a denotes the number of agents revealed by the algorithm for a . i.e.,

$$OPT \geq ALG - 2$$

for each agent a matched to its revealed post

- Number of queries: $OPT \geq ALG - 2$ for each agent
- This leads us to a competitive ratio of 4, which means that ALG asks at most 4 times the number of queries OPT asks in the worst case.

Competitive Ratio

$$\alpha = \frac{ALG}{OPT} \leq \frac{\sum_{a \in A'} q_a + n - 1}{\sum_{a \in A'} \max(1, q_a - 2)} = \frac{\sum_{a \in A'} (q_a + 1)}{\sum_{a \in A'} \max(1, q_a - 2)} \leq 4$$

- α is at most 4 at $q_a = 3$.

SIMULATOR

- Developed a real-time interactive simulator to visualize the elicitation of Necessarily Rank-Maximal (NRM) Matchings via the RASC query protocol.
- Algorithmic states are visualized:
 - Dynamic bipartite graph updates showing vertex availability and matching progression.
 - A robust logging system that captures query/response cycles and enforces strict input logic against manipulation.
 - Live state tracking of the algorithm's in-memory dictionary and active agent preferences.
- Deployed the full-stack application (Python, FastAPI, JavaScript) on Render, utilizing WebSockets to handle real-time, low-latency graph updates.
- Implemented library modules *graph_utils* and *edmonds_gallai* using Networkx to support backend

Attachments

- RASC-based NRM Simulator
- Algorithm
- Report
- Github repository

Thank you!